

IO_DIO.h File Reference

IO Driver functions for Digital Input/Output. [More...](#)

```
#include "IO_Error.h"
```

Data Structures

struct **IO_DIO_LIMITS**
Voltage limits for digital inputs. [More...](#)

struct **IO_DO_SAFETY_CONF**
Safety configuration for the digital outputs. [More...](#)

Macros

Pull up / Pull down configuration for digital inputs

Configuration of the pull up or pull down resistors on the digital inputs. These defines can be used for the `pupd` parameter of the function `IO_DI_Init()`.

```
#define IO_DI_NO_PULL 0x00U
```

```
#define IO_DI_PU_10K 0x01U
```

```
#define IO_DI_PD_10K 0x02U
```

Functions

IO_ErrorType **IO_DI_Init** (**ubyte1** di_channel, **ubyte1** pupd, const **IO_DIO_LIMITS** *const limits)
Setup a digital input. [More...](#)

IO_ErrorType **IO_DO_Init** (**ubyte1** do_channel, **bool** diagnostic, const **IO_DO_SAFETY_CONF** *const safety_conf)
Setup a digital output. [More...](#)

IO_ErrorType **IO_DI_DeInit** (**ubyte1** di_channel)
Deinitializes a digital input. [More...](#)

IO_ErrorType **IO_DO_DeInit** (**ubyte1** do_channel)
Deinitializes a digital output. [More...](#)

IO_ErrorType **IO_DI_Get** (**ubyte1** di_channel, **bool** *const di_value)
Gets the value of a digital input. [More...](#)

IO_ErrorType **IO_DO_Set** (**ubyte1** do_channel, **bool** do_value)
Sets the value of a digital output. [More...](#)

IO_ErrorType **IO_DO_GetCur** (**ubyte1** do_channel, **ubyte2** *const current, **bool** *const fresh)
Returns the measured current of a digital output. [More...](#)

IO_ErrorType IO_DO_GetVoltage (**ubyte1** do_channel, **ubyte2** *const voltage, **bool** *const fresh)

Returns the measured voltage of a digital output. [More...](#)

IO_ErrorType IO_DO_ResetProtection (**ubyte1** do_channel, **ubyte1** *const reset_cnt)

Reset the output protection for a digital output. [More...](#)

Detailed Description

IO Driver functions for Digital Input/Output.

Contains all service functions for the digital in/outputs.

Note

The digital inputs reflect the current status of the input at the point in time where the function is called.

The digital outputs [IO_DO_00](#) .. [IO_DO_15](#) are controlled over SPI shift registers, therefore the outputs will be periodically updated with a cycle of 1ms.

DIO-API Usage:

- [Examples for DIO API functions](#)

Digital output protection

Each digital output is individually protected against overcurrent. Whenever a fatal overcurrent situation is detected on any safe or unsafe digital output, the output is disabled by software. Note that disabled by software means that the output is set to low but the state of the safety switch (if configured) remains unchanged.

When entering the protection state, the digital output has to remain in this state for the given wait time. After this wait time the digital output can be reenabled via function [IO_DO_ResetProtection\(\)](#). Note that the number of reenabling operations for a single digital output is limited to 10. Refer to function [IO_DO_ResetProtection\(\)](#) for more information on how to reenable a digital output.

Detailed information about fatal overcurrent situations, the reaction time and wait time are listed in the following table.

Digital output	Fatal overcurrent situation	Latest disabled within	Wait time
IO_DO_00 .. IO_DO_15 , IO_DO_52 .. IO_DO_59	3.5A if board temperature > 85°C and persistent for > 100ms	12ms	1s
	4.7A if board temperature ≤ 85°C and persistent for > 100ms		
	5 samples above 7A		

	the switch has already switched off by itself		
IO_DO_16 .. IO_DO_51	3.1A if board temperature > 85°C and persistent for > 100ms	6ms	10s
	4.1A if board temperature ≤ 85°C and persistent for > 100ms		
	1 sample above 5.5A		

Digital output protection overview

DIO Code Examples

Examples for using the DIO API

DIO initialization example

```
IO_DI_Init(IO_DI_00,           // digital input
           IO_DI_NO_PULL,     // fixed pull resistor
           NULL);             // no limit configuration (only for analog
                             channels)

IO_DO_Init(IO_DO_18,           // digital output
           FALSE,             // diagnostic disabled
           NULL);             // no safety configuration
```

DIO task function example

```
bool di_val_0;

IO_DI_Get(IO_DI_00,           // read value of digital input
          &di_val_0);

IO_DO_Set(IO_DO_18,           // set digital output value
          TRUE);
```

Macro Definition Documentation

```
#define IO_DI_NO_PULL 0x00U
```

fixed pull resistor

```
#define IO_DI_PD_10K 0x02U
```

10 kOhm pull down

```
#define IO_DI_PU_10K 0x01U
```

10 kOhm pull up

Function Documentation

IO_ErrorType IO_DI_DeInit (ubyte1 di_channel)

Deinitializes a digital input.

Parameters

di_channel Digital input:

- IO_DI_00 .. IO_DI_35
- IO_DI_36 .. IO_DI_47
- IO_DI_48 .. IO_DI_55
- IO_DI_56 .. IO_DI_63
- IO_DI_64 .. IO_DI_71
- IO_DI_72 .. IO_DI_79
- IO_DI_80 .. IO_DI_87
- IO_DI_88 .. IO_DI_95

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_CH_CAPABILITY	the given channel is not a digital input channel
IO_E_INVALID_PARAMETER	internally an invalid parameter has been passed
IO_E_UNKNOWN	an unknown error occurred


```
IO_ErrorType IO_DI_Get ( ubyte1      di_channel,
                        bool *const di_value
                        )
```

Gets the value of a digital input.

Parameters

di_channel Digital input:

- IO_DI_00 .. IO_DI_35
- IO_DI_36 .. IO_DI_47
- IO_DI_48 .. IO_DI_55
- IO_DI_56 .. IO_DI_63
- IO_DI_64 .. IO_DI_71
- IO_DI_72 .. IO_DI_79
- IO_DI_80 .. IO_DI_87
- IO_DI_88 .. IO_DI_95

[out] **di_value** Input value:

- **TRUE**: High level
- **FALSE**: Low level

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_SHORT_BAT	the measured analog value is above the valid high band
IO_E_SHORT_GND	the measured analog value is below the valid low band
IO_E_INVALID_VOLTAGE	the measured analog value is in between the valid bands for high and low
IO_E_INVALID_CHANNEL_ID	the channel id does not exist
IO_E_NULL_POINTER	NULL pointer has been passed
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_CH_CAPABILITY	the given channel is not a digital input channel
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_REFERENCE	the internal reference voltage is out of range
IO_E_UNKNOWN	an unknown error occurred

Remarks

This function returns the value of the digital input pins at the time when the function is called for:

- `IO_DI_00 .. IO_DI_35`
- `IO_DI_36 .. IO_DI_47`

This function returns the value of the digital input pins at the time when the last AD value was sampled for.

- `IO_DI_48 .. IO_DI_55`
- `IO_DI_56 .. IO_DI_63`
- `IO_DI_64 .. IO_DI_71`
- `IO_DI_72 .. IO_DI_79`
- `IO_DI_80 .. IO_DI_87`
- `IO_DI_88 .. IO_DI_95`

The error codes `IO_E_SHORT_BAT`, `IO_E_SHORT_GND` and `IO_E_INVALID_VOLTAGE` are only returned for `IO_DI_56 .. IO_DI_63`, `IO_DI_64 .. IO_DI_71`, `IO_DI_72 .. IO_DI_79`, `IO_DI_80 .. IO_DI_87` and `IO_DI_88 .. IO_DI_95` and only if corresponding limits are defined.


```

IO_ErrorType IO_DI_Init ( ubyte1          di_channel,
                          ubyte1          pupd,
                          const IO_DIO_LIMITS *const limits
                          )

```

Setup a digital input.

Parameters

di_channel Digital input:

- `IO_DI_00 .. IO_DI_35`
- `IO_DI_36 .. IO_DI_47`
- `IO_DI_48 .. IO_DI_55`
- `IO_DI_56 .. IO_DI_63`
- `IO_DI_64 .. IO_DI_71`
- `IO_DI_72 .. IO_DI_79`
- `IO_DI_80 .. IO_DI_87`
- `IO_DI_88 .. IO_DI_95`

pupd

Pull up/down configuration:

- `IO_DI_NO_PULL`: fixed pull resistor
- `IO_DI_PU_10K`: Pull up 10 kOhm
- `IO_DI_PD_10K`: Pull down 10 kOhm

[in] **limits**

Voltage limits for low/high-levels. If `NULL`, default limits will be used. See `IO_DIO_LIMITS` for details.

Returns

`IO_ErrorType`:

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist or is not available on the used ECU variant
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a digital input channel or the used ECU variant does not support this function
<code>IO_E_CHANNEL_BUSY</code>	the digital output channel is currently used by another function
<code>IO_E_INVALID_PARAMETER</code>	parameter is out of range
<code>IO_E_INVALID_LIMITS</code>	the given voltage limits are not valid
<code>IO_E_FPGA_NOT_INITIALIZED</code>	the FPGA has not been initialized
<code>IO_E_DRIVER_NOT_INITIALIZED</code>	

the common driver init function has not been called before

IO_E_UNKNOWN

an unknown error occurred

Remarks

- The supported features depend on the selected channel:
 - `IO_DI_00 .. IO_DI_35`:
 - `pupd` and `limits` ignored
 - `IO_DI_36 .. IO_DI_47`:
 - `pupd`: `IO_DI_PU_10K` or `IO_DI_PD_10K`
 - `IO_DI_48 .. IO_DI_55`:
 - `pupd` and `limits` ignored
 - `IO_DI_56 .. IO_DI_63`:
 - `limits`: Voltage limits for low/high-levels
 - `IO_DI_64 .. IO_DI_71`:
 - `limits`: Voltage limits for low/high-levels
 - `IO_DI_72 .. IO_DI_79`:
 - `pupd`: `IO_DI_PU_10K` or `IO_DI_PD_10K`
 - `limits`: Voltage limits for low/high-levels
 - `IO_DI_80 .. IO_DI_87`:
 - `limits`: Voltage limits for low/high-levels
 - `IO_DI_88 .. IO_DI_95`:
 - `pupd`: `IO_DI_PU_10K` or `IO_DI_PD_10K`
 - `limits`: Voltage limits for low/high-levels
- The inputs `IO_DI_80 .. IO_DI_87` have a fixed pull up. Depending on the parameter `limits` switches to ground or BAT can be read.
- The inputs `IO_DI_00 .. IO_DI_35`, `IO_DI_36 .. IO_DI_47` and `IO_DI_48 .. IO_DI_55` have a fixed switching threshold of 2.5V.

Attention

The inputs `IO_DI_00 .. IO_DI_35` and `IO_DI_48 .. IO_DI_55` have a fixed pull up and are only suitable for switches to ground.

The inputs `IO_DI_56 .. IO_DI_63` and `IO_DI_64 .. IO_DI_71` are only suitable for switches to BAT.

IO_ErrorType IO_DO_DeInit (ubyte1 do_channel)

Deinitializes a digital output.

Parameters

do_channel Digital output:

- IO_DO_00 .. IO_DO_07
- IO_DO_08 .. IO_DO_15
- IO_DO_16 .. IO_DO_51
- IO_DO_52 .. IO_DO_59

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_CH_CAPABILITY	the given channel is not a digital output channel
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

```
IO_ErrorType IO_DO_GetCur ( ubyte1      do_channel,
                           ubyte2 *const current,
                           bool *const  fresh
                           )
```

Returns the measured current of a digital output.

Parameters

do_channel Digital output:

- `IO_DO_00 .. IO_DO_07`
- `IO_DO_08 .. IO_DO_15`
- `IO_DO_16 .. IO_DO_51`
- `IO_DO_52 .. IO_DO_59`

[out] **current** Measured current in mA Range: 0..7500 (0A .. 7.500A)

[out] **fresh** Indicates if new values are available since the last call.

- **TRUE**: Value in "current" is valid
- **FALSE**: No new value available.

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_NULL_POINTER	a NULL pointer has been passed to the function
IO_E_CH_CAPABILITY	the given channel is not a digital output channel
IO_E_CM_CALIBRATION	the zero current calibration failed
IO_E_REFERENCE	the internal reference voltage is out of range
IO_E_UNKNOWN	an unknown error occurred

Note

The error code **IO_E_CM_CALIBRATION** is only returned for `IO_DO_16 .. IO_DO_51`

Remarks

If there is no new current value available (for example the function `IO_DO_GetCur()` gets called more frequently than the AD sampling) the flag `fresh` will be set to **FALSE**.

```
IO_ErrorType IO_DO_GetVoltage ( ubyte1      do_channel,
                                ubyte2 *const voltage,
                                bool *const  fresh
                                )
```

Returns the measured voltage of a digital output.

Parameters

do_channel Digital output:

- `IO_DO_00 .. IO_DO_07`
- `IO_DO_52 .. IO_DO_59`

[out] **voltage** Measured voltage in mV. Range: 0..32000 (0V..32.000V)

[out] **fresh** Indicates if new values are available since the last call.

- `TRUE`: Value in "current" is valid
- `FALSE`: No new value available.

Returns

IO_ErrorType:

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_INVALID_CHANNEL_ID</code>	the channel id does not exist
<code>IO_E_NULL_POINTER</code>	a NULL pointer has been passed to the function
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given channel is not configured
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a digital output channel
<code>IO_E_REFERENCE</code>	the internal reference voltage is out of range

Remarks

If there is no new voltage value available (for example the function `IO_DO_GetVoltage()` gets called more frequently than the AD sampling) the flag `fresh` will be set to `FALSE`.


```

IO_ErrorType IO_DO_Init ( ubyte1 do_channel,
                          bool diagnostic,
                          const IO_DO_SAFETY_CONF *const safety_conf
                          )

```

Setup a digital output.

Parameters

do_channel Digital output:

- `IO_DO_00 .. IO_DO_07`
- `IO_DO_08 .. IO_DO_15`
- `IO_DO_16 .. IO_DO_51`
- `IO_DO_52 .. IO_DO_59`

diagnostic Output configuration:

- `TRUE`: diagnostic pull-up enabled
- `FALSE`: diagnostic pull-up disabled

[in] **safety_conf** Relevant safety configurations for the checker modules. The following DO channels can be configured safety relevant:

- `IO_DO_00 .. IO_DO_07`

Returns

IO_ErrorType:

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist or is not available on the used ECU variant
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a digital output channel or the used ECU variant does not support this function
<code>IO_E_CHANNEL_BUSY</code>	the digital output channel is currently used by another function
<code>IO_E_CM_CALIBRATION</code>	the zero current calibration failed
<code>IO_E_FPGA_NOT_INITIALIZED</code>	the FPGA has not been initialized
<code>IO_E_DRIVER_NOT_INITIALIZED</code>	the common driver init function has not been called before
<code>IO_E_INVALID_SAFETY_CONFIG</code>	an invalid safety configuration has been passed

IO_E_DRV_SAFETY_CONF_NOT_CONFIG	global safety configuration is missing
IO_E_DRV_SAFETY_CYCLE_RUNNING	the init function was called after the task begin function
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Note

The error code **IO_E_CM_CALIBRATION** is only returned for **IO_DO_16 .. IO_DO_51**

Remarks

- The digital output channels **IO_DO_00 .. IO_DO_15** are controlled over SPI shift registers. Therefore the outputs will be periodically updated with a cycle of 1ms.
- The digital output channels **IO_DO_16 .. IO_DO_51** are an alternative function to **IO_PWM_00 .. IO_PWM_35**.
- The digital output channels **IO_DO_52 .. IO_DO_59** are an alternative function to **IO_PVG_00 .. IO_PVG_07**.
- The parameter `diagnostic` is only applied to the channels **IO_DO_00 .. IO_DO_07** and **IO_DO_52 .. IO_DO_59**. If `diagnostic` is **TRUE**, the output can detect open load and short circuit. If it is **FALSE**, the output can not detect open load or short circuit. Select **FALSE** for loads with low current consumption like LEDs. With `diagnostic == FALSE` the pull up will be switched off.
- If `safety_conf != NULL`, a low side and high side channel have to be connected together. The internal checker modules check the given channels against the parameter in `safety_conf`. For more detail about each parameter look on the definition of **IO_DO_SAFETY_CONF**
- If `safety_conf != NULL`, the parameter `diagnostic` is forced to **TRUE** to allow diagnostics


```
IO_ErrorType IO_DO_ResetProtection ( ubyte1      do_channel,
                                     ubyte1 *const reset_cnt
                                   )
```

Reset the output protection for a digital output.

Parameters

do_channel Digital output:

- `IO_DO_00 .. IO_DO_07`
- `IO_DO_08 .. IO_DO_15`
- `IO_DO_16 .. IO_DO_51`
- `IO_DO_52 .. IO_DO_59`

[out] **reset_cnt** Protection reset counter. Indicates how often the application already reset the protection.

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist
<code>IO_E_FET_PROT_NOT_ACTIVE</code>	no output FET protection is active
<code>IO_E_FET_PROT_PERMANENT</code>	the output FET is protected permanently because it was already reset more than 10 times
<code>IO_E_FET_PROT_WAIT</code>	the output FET protection can not be reset, as the wait time of 1s is not already passed
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given channel is not configured
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a digital output channel
<code>IO_E_UNKNOWN</code>	an unknown error occurred

Attention

The protection can be reset 10 times, afterwards the output will remain permanently protected

Note

- After entering the output protection, a certain time has to pass before the output protection can be reset:
 - 1s for `IO_DO_00 .. IO_DO_07`
 - 1s for `IO_DO_08 .. IO_DO_15`
 - 1s for `IO_DO_52 .. IO_DO_59`

- 10s for `IO_DO_16 .. IO_DO_51`

- This function will not set the output back again to high. After calling this function the output has to be set to the intended level with `IO_DO_Set()`

Remarks

If the parameter `reset_cnt` is `NULL`, the parameter is ignored. The parameter `reset_cnt` returns the number of resets already performed.


```
IO_ErrorType IO_DO_Set ( ubyte1 do_channel,
                        bool do_value
                        )
```

Sets the value of a digital output.

Parameters

do_channel Digital output:

- `IO_DO_00 .. IO_DO_07`
- `IO_DO_08 .. IO_DO_15`
- `IO_DO_16 .. IO_DO_51`
- `IO_DO_52 .. IO_DO_59`

do_value Output value:

- `TRUE`: High level
- `FALSE`: Low level

Returns

`IO_ErrorType`:

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_INVALID_CHANNEL_ID</code>	the channel id does not exist
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given channel is not configured
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a digital output channel
<code>IO_E_STARTUP</code>	the output is in the startup phase
<code>IO_E_NO_DIAG</code>	no diagnostic feedback available
<code>IO_E_OPEN_LOAD</code>	open load has been detected
<code>IO_E_SHORT_GND</code>	short circuit has been detected
<code>IO_E_SHORT_BAT</code>	short to battery voltage has been detected
<code>IO_E_OPEN_LOAD_OR_SHORT_BAT</code>	open load or short to battery voltage has been detected
<code>IO_E_FET_PROT_ACTIVE</code>	the output FET protection is active and has set the output to low. It can be reset with <code>IO_DO_ResetProtection()</code> after the wait time is passed
<code>IO_E_FET_PROT_REENABLE</code>	the output FET protection is ready to be reset with <code>IO_DO_ResetProtection()</code>
<code>IO_E_FET_PROT_PERMANENT</code>	

	the output FET is protected permanently because it was already reset more than 10 times. The output will remain low
IO_E_SAFETY_SWITCH_DISABLED	The safety switch of the corresponding output is disabled. The output is currently forced to low.
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_SAFE_STATE	the digital output channel is in a safe state
IO_E_UNKNOWN	an unknown error occurred

Remarks

- The digital output channels **IO_DO_00** .. **IO_DO_15** are controlled over SPI shift registers. Therefore the outputs will be periodically updated with a cycle of 1ms.
- The digital output channels **IO_DO_16** .. **IO_DO_51** are an alternative function to **IO_PWM_00** .. **IO_PWM_35**.
- The digital output channels **IO_DO_52** .. **IO_DO_59** are an alternative function to **IO_PWD_00** .. **IO_PWD_11**.